

M10 / PROTECTED ANSWERS

M10 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	5
Open after a saved attempt only	5
DE10-01 Why containers exist	5
Expected reasoning/control	5
Professional boundary	5
DE10-02 Images and containers	5
Expected reasoning/control	5
Professional boundary	5
DE10-03 Writing a Dockerfile	5
Expected reasoning/control	5
Professional boundary	5
DE10-04 Ports	6
Expected reasoning/control	6
Professional boundary	6
DE10-05 Volumes and persistence	6
Expected reasoning/control	6
Professional boundary	6
DE10-06 Networks and service communication	6
Expected reasoning/control	6
Professional boundary	6
DE10-07 Compose, configuration and secrets	6

Expected reasoning/control	6
Professional boundary	6
DE10-08 Why orchestration exists	7
Expected reasoning/control	7
Professional boundary	7
DE10-09 Airflow architecture and UI	7
Expected reasoning/control	7
Professional boundary	7
DE10-10 DAGs, tasks and dependencies	7
Expected reasoning/control	7
Professional boundary	7
DE10-11 TaskFlow, operators and task communication	7
Expected reasoning/control	7
Professional boundary	8
DE10-12 Scheduling and data intervals	8
Expected reasoning/control	8
Professional boundary	8
DE10-13 Retries, timeouts and failure behavior	8
Expected reasoning/control	8
Professional boundary	8
DE10-14 Parameters and task communication	8
Expected reasoning/control	8
Professional boundary	8

DE10-15 Connections, variables and secrets	9
Expected reasoning/control	9
Professional boundary	9
DE10-16 Sensors and waiting	9
Expected reasoning/control	9
Professional boundary	9
DE10-17 Logs and debugging	9
Expected reasoning/control	9
Professional boundary	9
DE10-18 Catchup, backfills and reprocessing	9
Expected reasoning/control	9
Professional boundary	10
DE10-19 Integration and production-style orchestration	10
Expected reasoning/control	10
Professional boundary	10

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open after a saved attempt only

DE10-01 | Why containers exist

Expected reasoning/control

On a learner PC with Docker working, hello-world proves the engine can pull an image and start a container. `docker image ls` shows stored images; `docker ps -a` shows container instances. These commands require a working Docker engine on the learner PC. Save their real output there; static course checks do not substitute for runtime evidence.

Professional boundary

Containers improve reproducibility only when the build is reproducible, the configuration boundary is explicit, and persistent state is handled separately. A containerized pipeline can still be non-idempotent, insecure or logically wrong.

DE10-02 | Images and containers

Expected reasoning/control

The image remains after a short-lived container exits. A failed run caused by a deliberately invalid URL does not imply the image vanished. Re-running with corrected runtime configuration creates a new container from the same image.

Professional boundary

Promotion usually moves immutable image candidates between environments while environment-specific configuration changes outside the image. This makes rollback and audit much easier than hand-editing running machines.

DE10-03 | Writing a Dockerfile

Expected reasoning/control

The image should build from the documented context and report appuser as the runtime user. ``docker history`` should show the build steps. The build is not considered proven in this artifact environment because Docker is absent here; the Dockerfile is statically checked instead.

Professional boundary

A professional Dockerfile makes the runtime boundary reviewable: known base, minimal context, deterministic dependency install, explicit work directory, non-root user and a clear foreground process.

DE10-04 | Ports

Expected reasoning/control

`docker compose ps` should show a host mapping similar to `0.0.0.0:8090->8000/tcp`. Changing `SOURCE_PORT` changes only the host entry point; the service-to-service port remains 8000.

Professional boundary

Expose only interfaces that need host/external access. Databases and internal services often communicate on private container networks without publishing every port to the host.

DE10-05 | Volumes and persistence

Expected reasoning/control

After ordinary down and up, the named volume still exists. After `down -v`, it is removed. The source bind mount remains on the host because it was never Docker-managed volume data.

Professional boundary

State ownership is an architectural decision. Volumes survive container replacement but do not by themselves provide backup, replication, retention or corruption recovery.

DE10-06 | Networks and service communication

Expected reasoning/control

With both services on the same Compose network, the worker can resolve source. The host-facing URL still uses localhost and the published port. The internal URL uses service name plus container port.

Professional boundary

Stable service discovery plus health/readiness checks is safer than coding around ephemeral addresses. Network boundaries also help you avoid publishing internal-only services to the host.

DE10-07 | Compose, configuration and secrets

Expected reasoning/control

`docker compose config` should resolve variables and show service/network/volume structure before any container starts. The worker should eventually write one report based on the served CSV. A clean clone should need only documented environment values, not hidden machine state.

Professional boundary

Version-control safe structure and names; inject environment-owned values at runtime; store real secrets in a secrets mechanism appropriate to the deployment environment. Reproducibility means a clean environment can rebuild the same topology from documented inputs.

DE10-08 | Why orchestration exists

Expected reasoning/control

Three partitions should produce row counts 4, 3 and 5. Re-running should overwrite the same partition result with the same business totals rather than append duplicate logical output. That behavior is what later makes backfill/retry discussion concrete.

Professional boundary

Orchestration adds controlled execution semantics around testable units. A good DAG is a readable dependency model, not a dumping ground for every line of business code.

DE10-09 | Airflow architecture and UI

Expected reasoning/control

Airflow 3.3.1 should be reported on the pinned learner environment, the Dag should appear without import errors, and one test/manual run should expose task/run state in the UI. These are pending native-runtime proofs, not release claims made from this container.

Professional boundary

Operational diagnosis improves when you name the layer first: definition/parse, scheduling, task execution, dependency/service, or data correctness. Airflow gives state; your engineering still has to interpret it.

DE10-10 | DAGs, tasks and dependencies

Expected reasoning/control

The graph is linear for the base lab: extract -> validate -> publish. If validation fails, publish should not run. This is a dependency safety property, not merely a visual preference.

Professional boundary

Graph design is operational design: task boundaries determine what can retry independently, what can run concurrently, what evidence is isolated and how failures block downstream side effects.

DE10-11 | TaskFlow, operators and task communication

Expected reasoning/control

Calling a decorated task while defining the Dag creates a task/XComArg relationship rather than immediately executing the function. In a real run Airflow stores task communication according to its XCom backend; our course keeps messages deliberately small.

Professional boundary

Use orchestration metadata for coordination; use data stores for data. This keeps scheduler/metadata systems small, makes retries clearer and avoids coupling workflow correctness to oversized XCom serialization.

DE10-12 | Scheduling and data intervals

Expected reasoning/control

Historical runs map to historical partitions deterministically. The Dag uses a timezone-aware start date. The course treats `data_interval_start/end` as the semantic boundary; it does not teach deprecated 'execution date means run-now time' intuition.

Professional boundary

Interval-correct workflows make backfills, retries and audits possible. Time semantics are part of data correctness, not a scheduler trivia question.

DE10-13 | Retries, timeouts and failure behavior

Expected reasoning/control

A simulated transient error records one failed attempt and then success. A deterministic invalid-row condition fails validation and blocks publish. Re-running a successful partition does not multiply logical output.

Professional boundary

Retry policy is part of side-effect design. Reliable orchestration distinguishes transient infrastructure failures from deterministic data/config defects and leaves enough evidence to diagnose both.

DE10-14 | Parameters and task communication

Expected reasoning/control

A manual trigger with an invalid Param should be rejected by Airflow's validation instead of reaching task code. A scheduled run uses the default. The value is small configuration, not a substitute for sending datasets between tasks.

Professional boundary

Parameters make controlled variation explicit. Keep them small, typed and business-readable; keep observed run facts in task outputs/metadata and bulk data in data stores.

DE10-15 | Connections, variables and secrets

Expected reasoning/control

A clean repository contains no real password/token literals. Moving between environments should replace connection/secret configuration without editing business DAG logic. Global Variables are not used as a database for per-run task data.

Professional boundary

Portable orchestration keeps code stable while environment-owned connection/secrets configuration changes behind stable identifiers. Least privilege and log sanitization remain required.

DE10-16 | Sensors and waiting

Expected reasoning/control

A sensor succeeds only when its condition is true; it should fail after the declared timeout rather than wait forever. Deferrable support depends on the provider/runtime and must be proven in the learner environment.

Professional boundary

Waiting is capacity planning. A correct sensor defines condition, cadence, timeout, resource mode and failure ownership; otherwise a quiet wait can be as operationally expensive as a failed task.

DE10-17 | Logs and debugging

Expected reasoning/control

A failure can be traced to a specific partition/attempt and compared with a later repaired attempt. The logs contain enough context to reproduce the problem but no password/token literals.

Professional boundary

Operationally useful logging connects workflow state to data evidence. Preserve failed attempts, sanitize sensitive values, and design logs so another engineer can diagnose without reproducing your entire local setup.

DE10-18 | Catchup, backfills and reprocessing

Expected reasoning/control

Each date has one deterministic output file. Reprocessing Sep 2 replaces/converges that partition rather than appending a second logical copy. A historical range can be reasoned about before using Airflow's live backfill commands.

Professional boundary

Historical recovery is a first-class reliability feature. Partition identity, side-effect idempotency and explicit range controls determine whether backfill restores trust or creates duplicate damage.

DE10-19 | Integration and production-style orchestration

Expected reasoning/control

Static/local QA proves the files are internally coherent and the simulator demonstrates deterministic partition behavior. It does NOT prove Docker Engine, Airflow scheduler/UI or DagRun behavior on the learner PC. Those remain explicit runtime checks before calling the module fully native-validated.

Professional boundary

M10 mastery means you can defend both layers: environment reproducibility and workflow reliability. The G04 gate tests fresh pipeline reasoning after this module; it is not passed by rereading the final lesson.